



SMART PARKING SOLUTION

ParkSense

Application de gestion intelligente de parkings
avec intégration IoT et validation administrative

Rapport Technique — Architecture & Conception

Version 2.0 — Juin 2026



Sommaire

01	Introduction	3
02	Architecture Technique	3
03	Modèle de Données (Base de données)	4
04	Diagramme de Cas d'Utilisation	5
05	Workflows	6
06	Captures d'Écran de l'Application	7
07	Guide d'Installation	8
08	Technologies Utilisées	9

1. Introduction

ParkSense est une application mobile de gestion intelligente de parkings développée avec **React Native** et **Expo**. Elle permet aux utilisateurs de visualiser en temps réel la disponibilité des places de parking, de réserver des places, et de naviguer vers le parking choisi. Le système intègre des capteurs IoT (ESP32) via le protocole MQTT pour mettre à jour automatiquement l'état d'occupation des places.

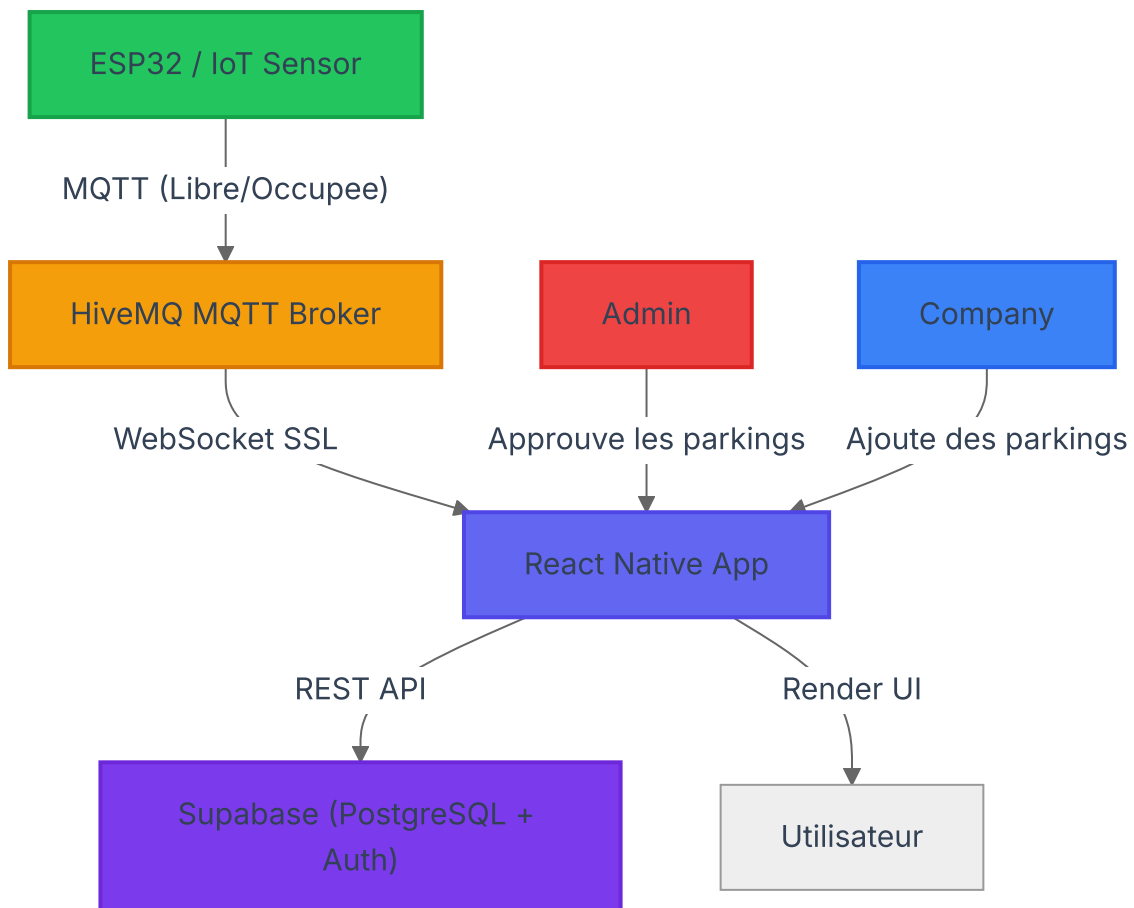
 **Nouveau dans la version 2.0 :** Gestion multi-rôles (Utilisateur / Entreprise / Admin), workflow d'approbation des parkings par les administrateurs, et tableau de bord entreprise.

Fonctionnalités principales

- Authentification sécurisée (Supabase Auth) avec session de 15 minutes
- Carte interactive des parkings avec disponibilité en temps réel
- Intégration IoT avec ESP32 et MQTT (simulateur web inclus)
- Inscription avec choix du rôle : Utilisateur ou Entreprise
- Ajout de parkings par les entreprises avec validation par l'administrateur
- Tableau de bord administrateur avec métriques et approbation des parkings
- Favoris, navigation, et historique de stationnement

2. Architecture Technique

L'architecture de ParkSense suit un modèle **3-tiers** avec des capteurs IoT, un backend cloud et une application mobile.

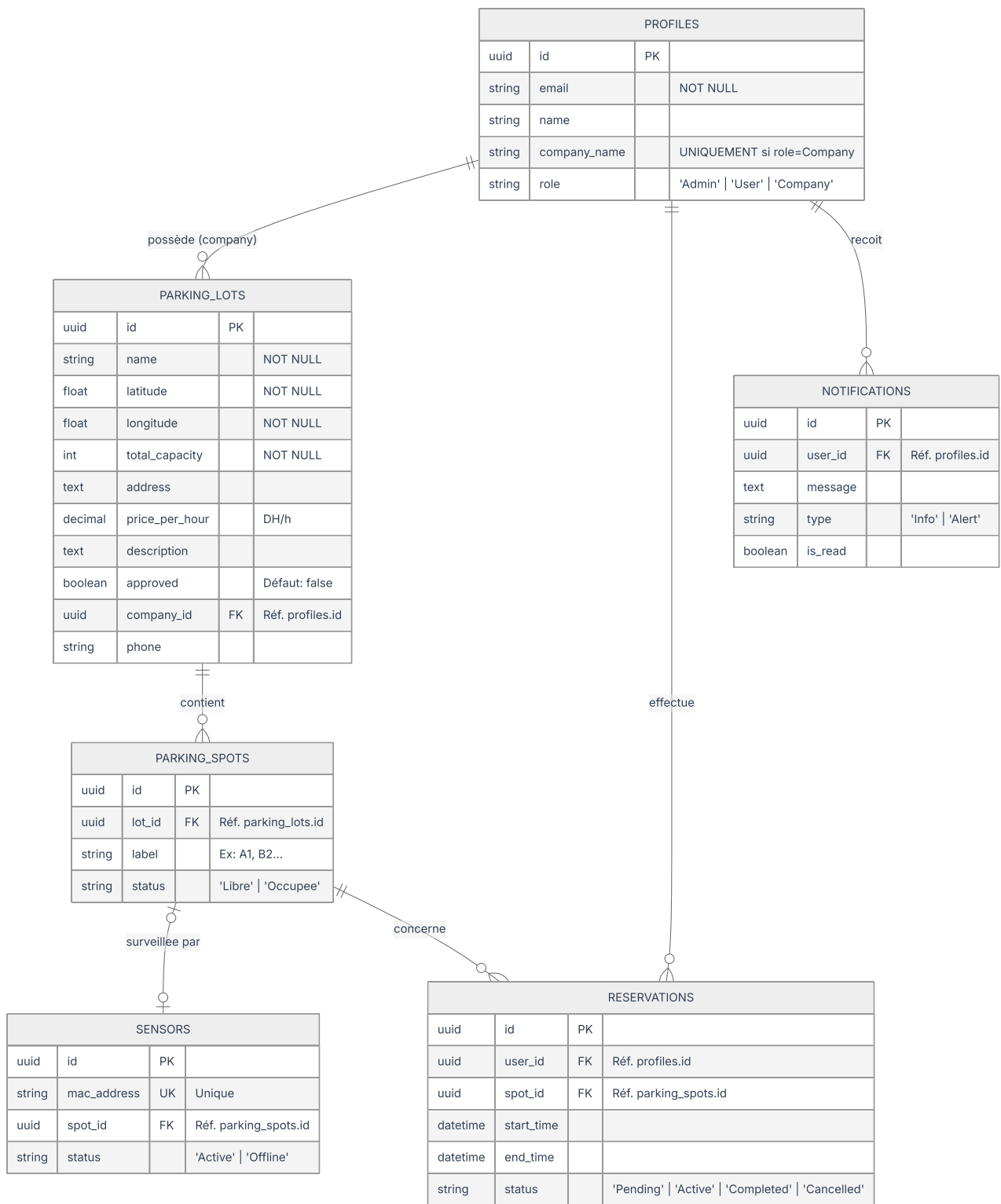


Flux des données

1. **Capteurs ESP32** : publie l'état (Libre/Occupee) sur le broker HiveMQ via MQTT
2. **Application mobile** : s'abonne aux topics MQTT et met à jour l'interface en temps réel
3. **Supabase** : stocke les données persistantes (parkings, utilisateurs, réservations)
4. **RLS (Row Level Security)** : sécurise l'accès aux données selon le rôle

3. Modèle de Données

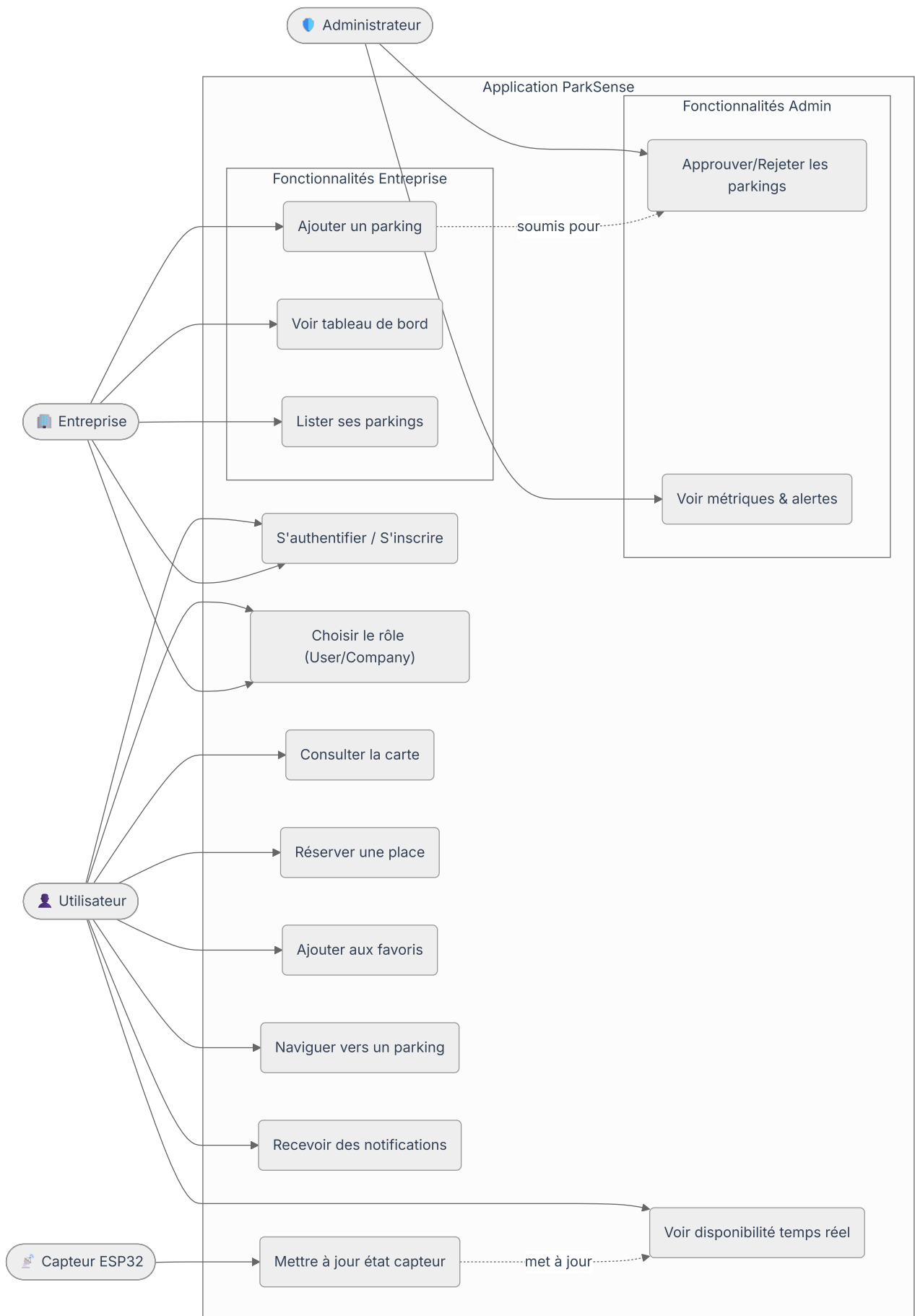
Diagramme entité-relation



Description des tables

Table	Rôle	Particularités
<code>profiles</code>	Profils utilisateurs	Rôles : User, Company, Admin ; <code>company_name</code> pour les entreprises
<code>parking_lots</code>	Parkings	<code>approved</code> contrôle l'affichage public ; <code>company_id</code> lie à l'entreprise propriétaire
<code>parking_spots</code>	Places individuelles	Statut binaire Libre/Occupee, lié à un lot
<code>sensors</code>	Capteurs IoT	Adresse MAC unique, lié à une place
<code>reservations</code>	Réservations	Lié à un utilisateur et une place
<code>notifications</code>	Notifications	Alertes push stockées en DB

4. Diagramme de Cas d'Utilisation

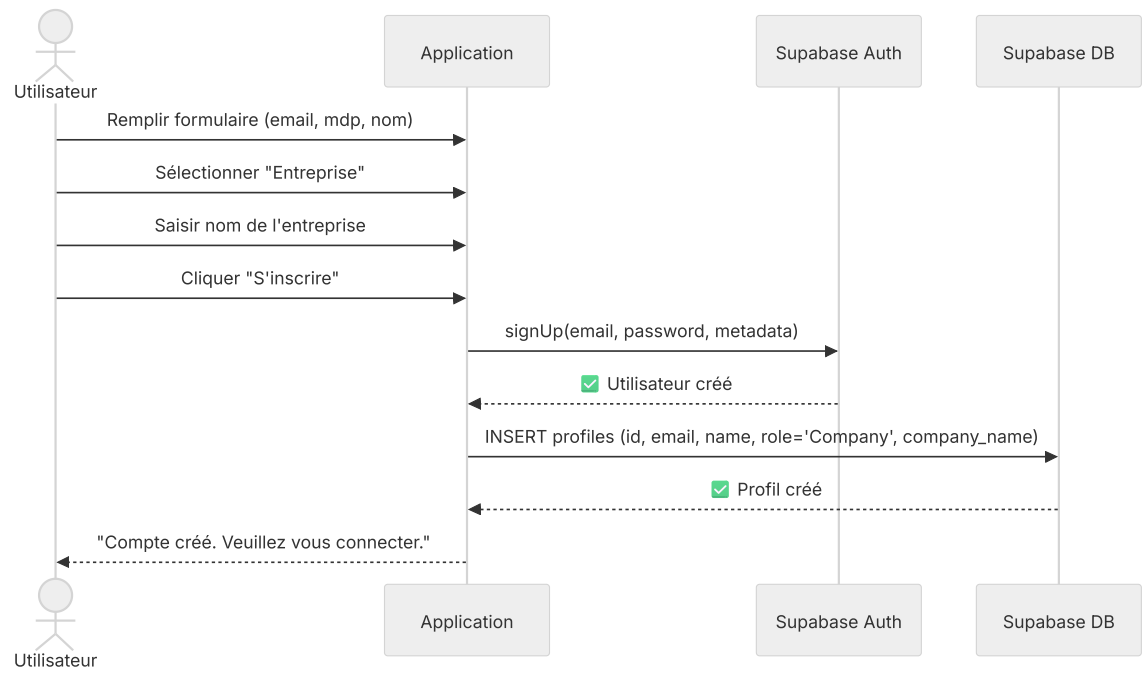


Acteurs

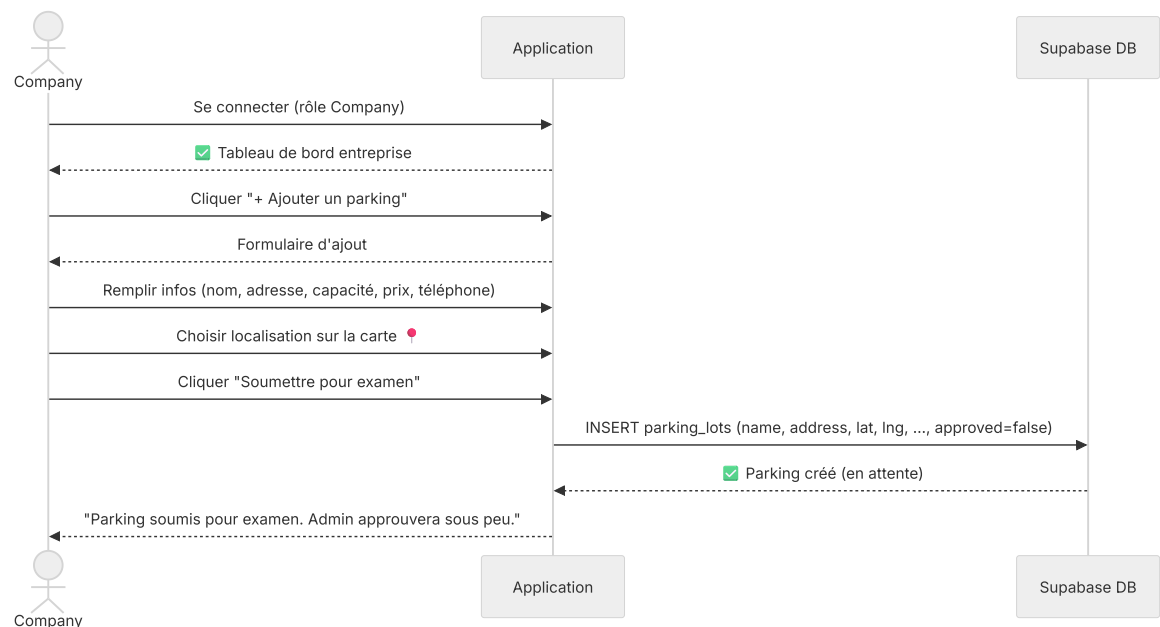
Acteur	Description	Cas d'utilisation
 Utilisateur	Personne cherchant une place de parking	Consulter la carte, réserver, naviguer, favoris
 Entreprise	Société de gestion de parkings	Ajouter des parkings, voir tableau de bord
 Administrateur	Superviseur du système	Approuver/rejeter les parkings, métriques
 Capteur ESP32	Capteur IoT physique	Publier l'état d'occupation via MQTT

5. Workflows

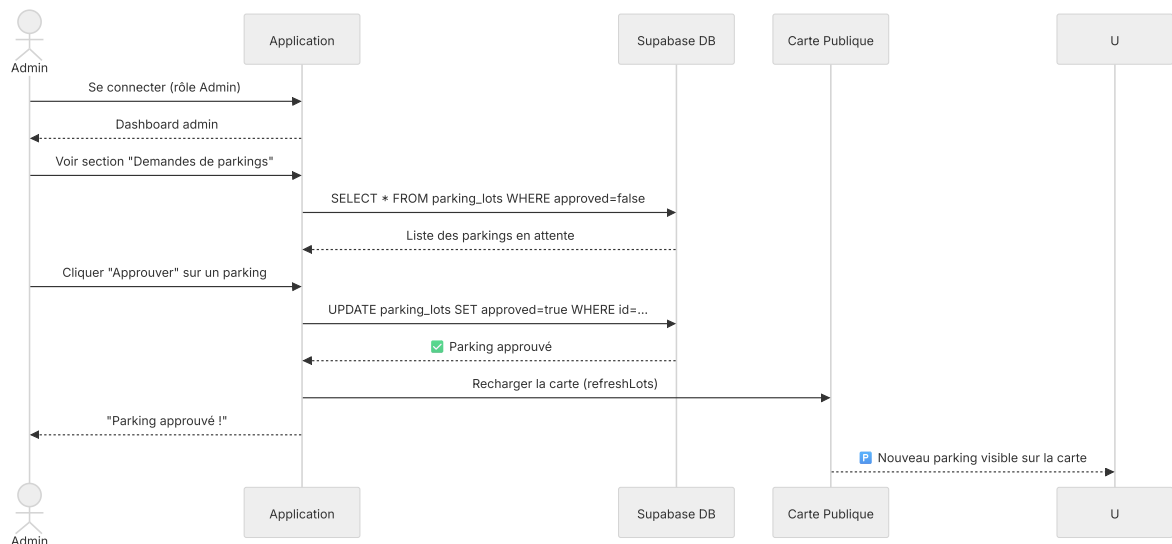
Workflow 1 : Inscription d'une entreprise



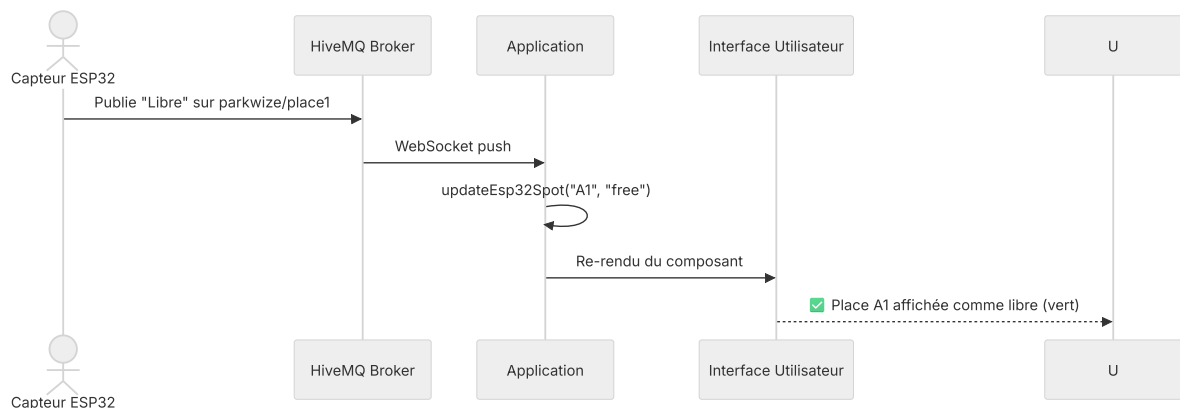
Workflow 2 : Ajout d'un parking par une entreprise



Workflow 3 : Approbation par l'administrateur

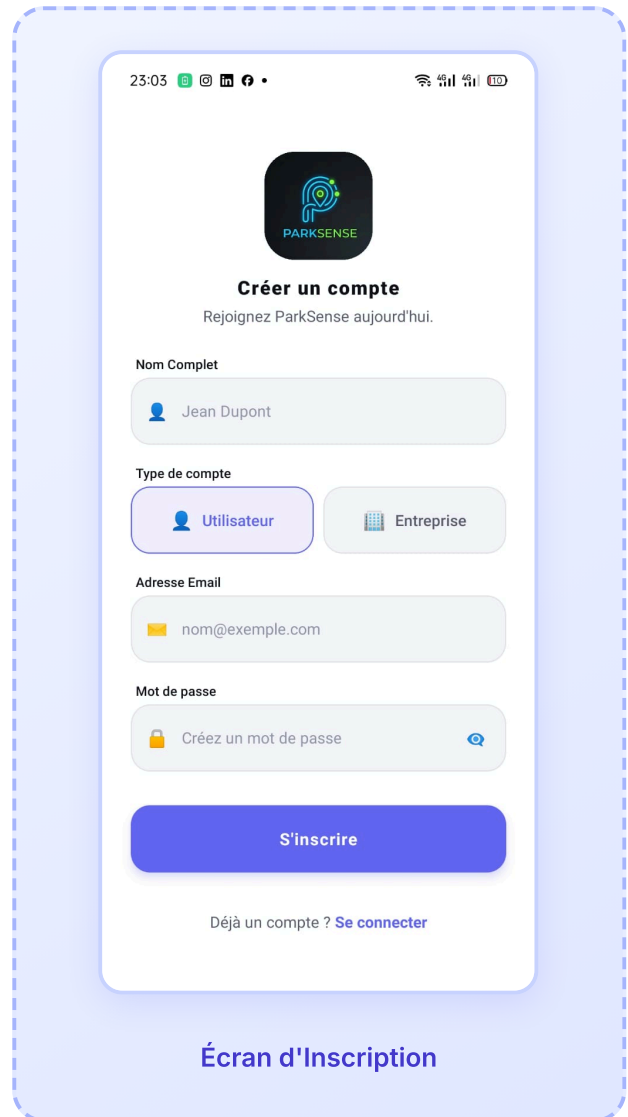
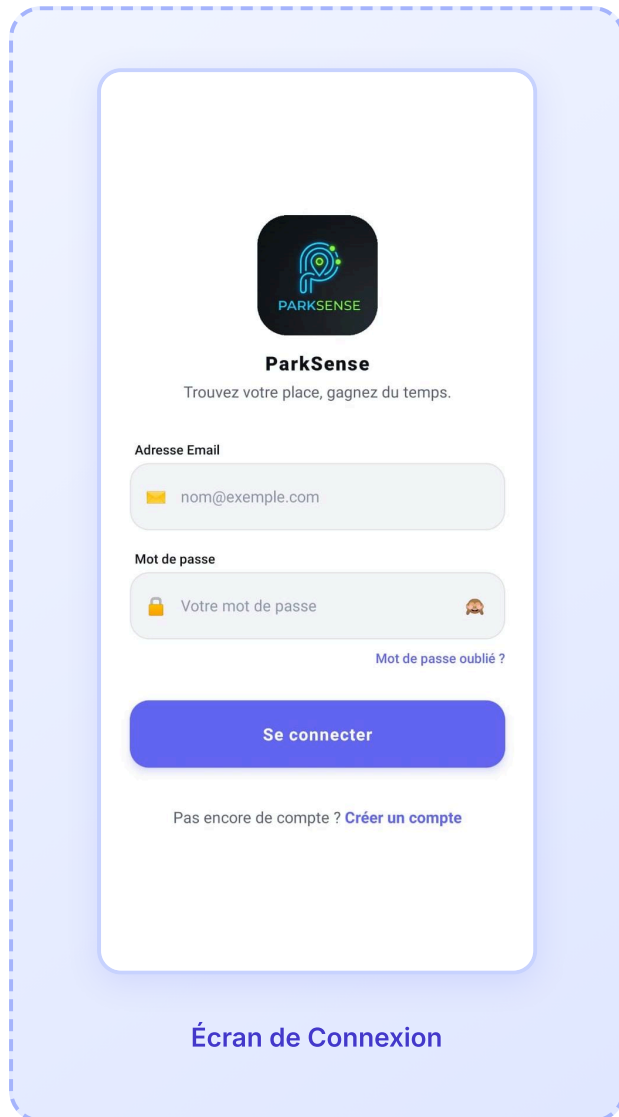


Workflow 4 : Mise à jour IoT en temps réel



6. Captures d'Écran de l'Application

Remplacez les placeholders ci-dessous par vos captures d'écran réelles. Les images sont à placer dans le dossier `captures d'ecran/` du projet.





Écran d'Accueil



Carte Interactive

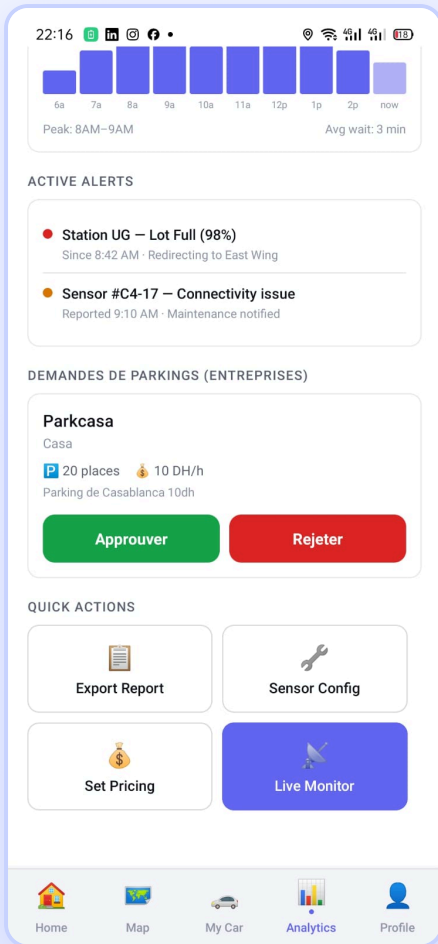


Tableau de Bord Entreprise

22:37

← Retour Ajouter un parking

Cacher la carte 33.554322, -7.573288

Casablanca

MAÂRIF المَعَارِيف

LAHRAOUIYINE الهراويين

AIN CHOCK عين الشق

Nouv Lahrac نوڤ لحرّاق

AÂROUF صوبدي

Capacité totale * 50

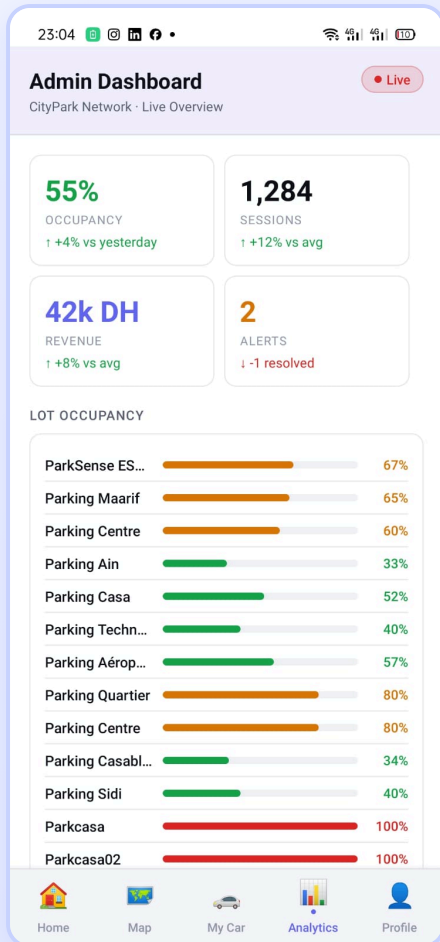
Prix (DH/h) 10

Description

Description de votre parking...

Soumettre pour examen

Ajout d'un Parking



Dashboard Administrateur

23:03



ParkSense

Trouvez votre place, gagnez du temps.

Adresse Email

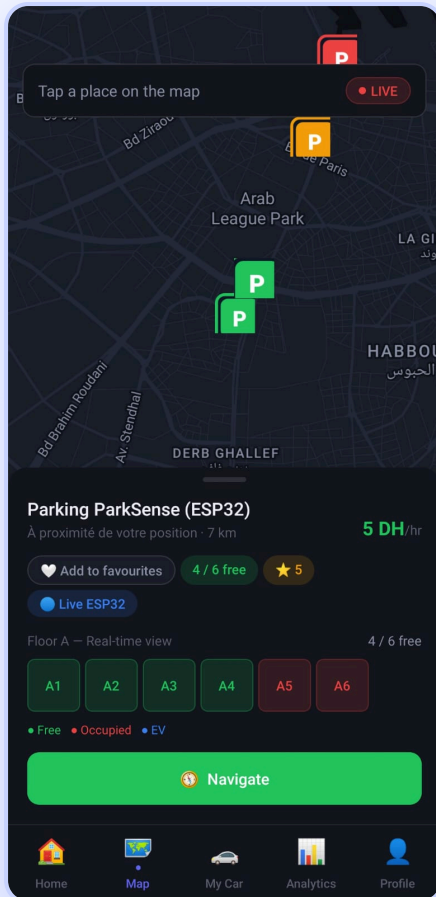
Mot de passe

[Mot de passe oublié ?](#)

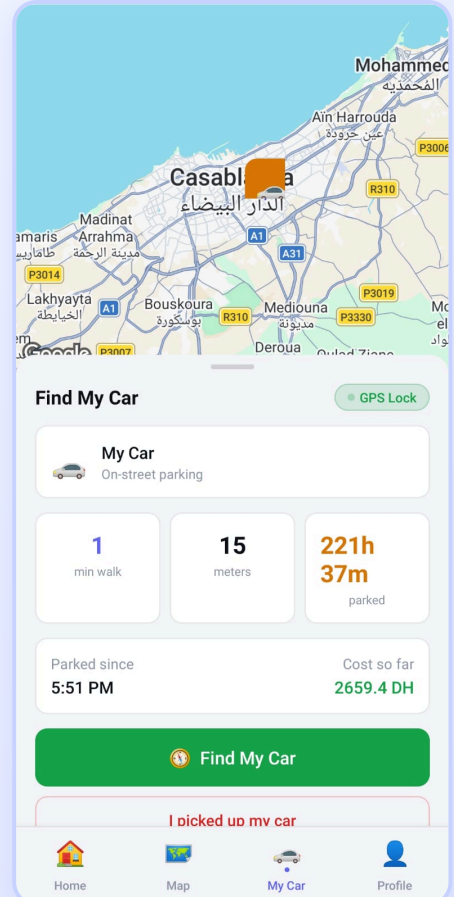
Se connecter

Pas encore de compte ? [Créer un compte](#)

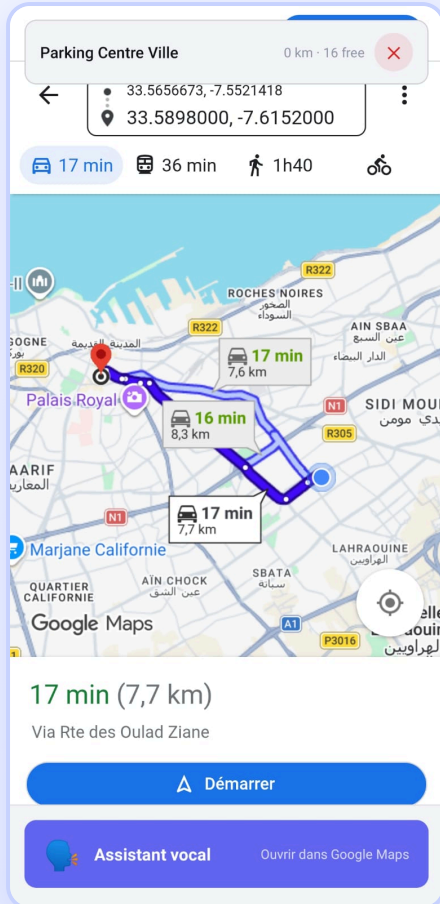
Profil Utilisateur



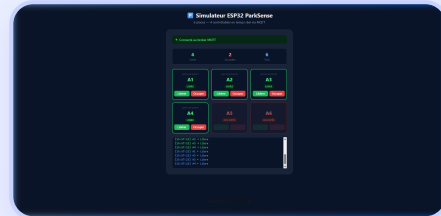
Détail d'un Parking



Retrouver sa Voiture



Navigation



Simulateur IoT (Web)

7. Guide d'Installation

Prérequis

- Node.js 18+ et npm
- Expo CLI (`npm install -g expo-cli`)
- Compte Supabase (gratuit)
- Appareil iOS/Android avec Expo Go, ou émulateur

Étapes d'installation

```
# 1. Cloner le dépôt
git clone https://github.com/SoufianeMajd/Parksense.git
cd Parksense

# 2. Installer les dépendances
npm install

# 3. Configurer l'environnement
# Créer un fichier .env avec les clés Supabase :
EXPO_PUBLIC_SUPABASE_URL=https://votre-projet.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=votre-cle-anonyme

# 4. Initialiser la base de données
# Exécuter dans l'éditeur SQL Supabase :
# 1. supabase_schema.sql

# 5. Lancer l'application
npx expo start
```

Création du compte administrateur

```
# 1. S'inscrire via l'app avec email: admin@parksense.com
# 2. Exécuter dans Supabase SQL :
INSERT INTO public.profiles (id, email, name, role)
SELECT id, email, raw_user_meta_data->>'name', 'Admin'
FROM auth.users WHERE email = 'admin@parksense.com'
ON CONFLICT (id) DO UPDATE SET role = 'Admin';
```

Ordre d'exécution SQL

Ordre	Fichier	Description
1	<code>supabase_schema.sql</code>	Création des tables et politiques RLS

8. Technologies Utilisées

Catégorie	Technologie	Version
Frontend	React Native + Expo	0.81 / 54
Langage	TypeScript	5.9
Navigation	React Navigation (Stack + Tabs)	6.x
Backend	Supabase (PostgreSQL + Auth)	-
Maps (native)	react-native-maps	1.20
Maps (web)	Leaflet + react-leaflet	1.9 / 5.0
IoT Protocol	MQTT (Paho client)	1.1
Stockage local	AsyncStorage	2.2
Notifications	Expo Notifications	-